



ZÉCODE × GITHUB

SmartHotel_Zehla · ZCC Platform

A BÍBLIA DO ZÉCODE

Manual Técnico Completo de GitOps,
Segurança e Automação GitHub

PAT VAULT

CI/CD ACTIONS

WEBHOOK + MIRROR

Versão 1.0 · Documento Técnico Oficial

Gerado em 18/08/2026

ZCC — Zélla Central Control

Índice

PARTE I — VISÃO ARQUITETURAL

1. Contexto e Justificativa

- 1.1. Por que GitHub como Plataforma de Operação?
- 1.2. Princípios de Segurança (Zero Trust)
- 1.3. Modelos de Autenticação Suportados

2. Tour Completo das Abas do GitHub

- 2.1. Aba Code (Código e Repositório)
- 2.2. Aba Issues (Issue Tracking)
- 2.3. Aba Pull Requests (Revisão de Código)
- 2.4. Aba Actions (CI/CD)
- 2.5. Aba Projects (Gestão de Projetos)
- 2.6. Aba Wiki (Documentação)
- 2.7. Aba Security (Segurança e Alertas)
- 2.8. Aba Insights (Analytics e Métricas)
- 2.9. Aba Settings (Configurações)
- 2.10. Aba Discussions (Fórum)

3. Comparativo de Métodos de Autenticação

- 3.1. PAT Clássico (NÃO RECOMENDADO)
- 3.2. PAT Fine-Grained (Recomendado)
- 3.3. GitHub App (Gold Standard)
- 3.4. Deploy Keys (Casos Específicos)
- 3.5. OIDC (Para Actions)

PARTE II — IMPLEMENTAÇÃO

4. Fase 1: PAT Vault Seguro

5. Fase 2: GitHub Client Service

6. Fase 3: Evolve-to-PR Pipeline

7. Fase 4: GitHub Action de PR Review

8. Fase 5: Webhook Listener

9. Fase 6: Mirror de Conta Corporativa

PARTE III — OPERAÇÃO

10. Troca e Rotação de Credenciais

11. Checklist de Implantação

12. Troubleshooting

13. Próximos Passos (Roadmap M1-M14)

PARTE I

Visão Arquitetural

1. Contexto e Justificativa

O ZéCode é o agente DEV FULL STACK interno do ZCC (Zélla Central Control) do projeto SmartHotel_Zehla. Inspirado no CodeRabbit.ai, ele evolui o código-fonte em paralelo ao Cérebro Zélla, que cuida das decisões em runtime. Para que o ZéCode atinja maturidade de produção, ele precisa operar o GitHub como uma plataforma de GitOps completa — não apenas ler diffs e comentar, mas criar branches, abrir Pull Requests, disparar workflows de CI/CD, rastrear issues, executar mirror de contas e responder a webhooks em tempo real.

Este documento é a referência técnica absoluta — a "Bíblia do ZéCode" — para que engenheiros possam implementar, operar, auditar e evoluir a integração GitHub × ZCC com segurança empresarial. Toda decisão arquitetural aqui documentada segue três princípios não-negociáveis: (1) princípio do menor privilégio, (2) zero downtime em rotações, (3) auditabilidade completa de cada ação realizada em nome do repositório.

1.1. Por que GitHub como Plataforma de Operação?

O GitHub deixou de ser apenas um repositório Git há muito tempo. Hoje ele é uma plataforma completa de engenharia de software que inclui CI/CD (Actions), gestão de projetos (Projects v2), segurança (CodeQL, Dependabot, Secret Scanning), issue tracking, discussions, wiki, e APIs REST e GraphQL extensivas. Para o ZéCode, usar o GitHub como plataforma significa herdar toda essa infraestrutura sem reinventar a roda — o agente foca em inteligência de código enquanto o GitHub provê a camada de orquestração, auditoria e collaboration.

A alternativa seria construir um sistema proprietário de revisão, fila de merges, tracking de issues e CI/CD. Isso consumiria meses de engenharia, reduziria a adoção pela equipe (que já conhece o GitHub) e nos obrigaria a manter infraestrutura que o GitHub já escala para milhões de repositórios. A escolha pelo GitHub é, portanto, estratégica e não apenas conveniente.

1.2. Princípios de Segurança (Zero Trust)

Toda credencial GitHub é tratada como material sensível de alto risco. Um PAT vazado pode dar acesso irrestrito a repositórios, permitir commits maliciosos, exfiltrar código-fonte, alterar workflows de CI/CD para injetar supply chain attacks, e comprometer todo o histórico do projeto. Por isso, adotamos uma postura Zero Trust:

Princípio	Implementação no ZéCode
Criptografia em repouso	AES-256-GCM com master key via env var (KMS)

Princípio	Implementação no ZéCode
Criptografia em trânsito	HTTPS/TLS 1.3 obrigatório, HSTS
Princípio do menor privilégio	Fine-Grained PAT com apenas escopos necessários
Rotação periódica	TTL máximo 90 dias, alerta a 14 dias antes
Auditabilidade completa	Toda leitura/uso do PAT registrado em PatAuditLog
Revogação imediata	API de revoke chama GitHub /applications/token
Sandbox de execução	Branches dedicadas, nunca commit em main/master
Rate limit enforcement	Backoff exponencial, respeita X-RateLimit-Remaining
Validação de webhooks	HMAC SHA-256 + IP allowlist do GitHub
Separação de ambientes	PATs distintos para prod/staging/dev

1.3. Modelos de Autenticação Suportados

O ZéCode suporta três modelos de autenticação GitHub, escolhidos conforme o caso de uso. A escolha correta é crítica: usar PAT clássico quando Fine-Grained bastaria é uma violação de least privilege; usar GitHub App para um único repositório pode ser over-engineering; usar Deploy Keys para operações de PR/Issue simplesmente não funciona (Deploy Keys são apenas para push/pull).

A tabela abaixo resume os três modelos. Detalhes completos de configuração estão no Capítulo 3.

Modelo	Granularidade	Rate Limit	Rotação	Caso de Uso
PAT Fine-Grained	Por repo + 40+ permissões	5000/h	Manual via Vault	Operações gerais (PR, issues, commits)
GitHub App	Por instalação + permissões	15000/h	JWT auto-rotate	Múltiplos repos, SaaS, multi-org
Deploy Keys	Por repo (read-only ou write)	N/A	Manual	Clone privado, push bot
OIDC Token	Por workflow run	Ilimitado*	Automático	Actions autenticando em cloud

* OIDC é emitido pelo GitHub Actions, não conta contra rate limit de API tradicional.

2. Tour Completo das Abas do GitHub

Para operar o GitHub com proficiência, o ZéCode precisa entender cada aba da interface do repositório e como ela se traduz em oportunidades de automação. Esta seção percorre todas as abas principais, descrevendo o que cada uma oferece e como o ZéCode a utiliza.

2.1. Aba Code (Código e Repositório)

A aba Code é o coração do repositório. Exibe a árvore de arquivos do branch atual, o README renderizado, o seletor de branch, o botão "Add file" (create/upload), e o botão "Code" que revela URL HTTPS, SSH, GitHub CLI, e a opção de abrir em Codespaces. Esta aba também mostra o número de commits, branches, tags, e a lista de arquivos com última mensagem de commit e autor.

Como o ZéCode usa esta aba:

O ZéCode consome a API de Contents para navegar pela árvore de arquivos sem precisar clonar o repositório inteiro. Isso é mais eficiente em largura de banda quando o agente só precisa ler um arquivo específico. Para operações de escrita, o ZéCode usa a API Git Database (trees, blobs, commits) que permite criar commits atômicos com múltiplos arquivos em uma única chamada — algo impossível via CLI sem `commit --amend`.

```
// Listar arquivos do repositório
GET /repos/{owner}/{repo}/contents/{path}?ref={branch}

// Criar commit com múltiplos arquivos (átomo)
POST /repos/{owner}/{repo}/git/trees
{
  "base_tree": "{sha_da_arvore_base}",
  "tree": [
    { "path": "src/foo.ts", "mode": "100644", "type": "blob", "content": "...", },
    { "path": "src/bar.ts", "mode": "100644", "type": "blob", "content": "...", }
  ]
}
```

2.2. Aba Issues (Issue Tracking)

A aba Issues é o sistema de tracking de bugs, features e tarefas. Cada issue suporta labels, milestones, assignees, projects vinculados, reactions, timeline de eventos, e referências cruzadas (auto-link via #ID). Issues podem ser criadas via API, via commit message (com "Fixes #123" fecha automaticamente a issue no merge do PR), ou via templates (.github/ISSUE_TEMPLATE/).

Como o ZéCode usa esta aba:

Quando o gap-detector identifica um problema crítico (ex.: ausência de validação de input em endpoint público), o ZéCode cria uma Issue com label "ze-code-finding" e "severity:critical". A issue inclui: descrição técnica, snippet de código com problema, sugestão de fix, e link para a RefactorSuggestion correspondente no banco de dados do ZCC. Quando um PR é aberto endereçando a issue, o ZéCode comenta nela com "Linked to PR #N — fix in progress", e quando o PR é merged, a issue é automaticamente fechada pelo GitHub (via "Fixes #M" no PR body).

```
// Criar issue
POST /repos/{owner}/{repo}/issues
{
  "title": "[ze-code-automated] Validação ausente em /api/bookings",
  "body": "## Descrição\n...",
  "labels": ["ze-code-automated", "severity:critical", "security"]
}
```

2.3. Aba Pull Requests (Revisão de Código)

A aba Pull Requests é onde a colaboração acontece. Cada PR exhibe o diff unificado, lista de commits, checks de CI/CD rodando, reviewers assignados, review comments inline, status de mergeability, e o botão merge (com opções: merge commit, squash, rebase). O GitHub também suporta draft PRs, PR templates (.github/PULL_REQUEST_TEMPLATE.md), auto-merge, e CODEOWNERS para auto-assign de reviewers por path.

Como o ZéCode usa esta aba:

O ZéCode atua em dois papéis neste fluxo: (1) como criador de PRs quando aplica refactorings detectados pelo gap-detector — sempre cria em branch dedicada (feat/ze-code/{slug}), com PR body estruturado incluindo justificativa técnica, métricas de confiança, e checklist de revisão; (2) como reviewer em PRs abertos por humanos — analisa o diff via /api/zcc/ze-code/github-review, gera comentários inline em linhas específicas, e publica um review resumido com APPROVE / REQUEST_CHANGES / COMMENT.

```
// Criar Pull Request
POST /repos/{owner}/{repo}/pulls
{
  "title": "[ze-code-automated] Refatorar validação de input /api/bookings",
  "head": "feat/ze-code/fix-validation-bookings",
  "base": "main",
  "body": "## Justificativa\nGap detectado em ...\n\nFixes #456",
  "draft": false
}

// Postar review com comentários inline
POST /repos/{owner}/{repo}/pulls/{pr_number}/reviews
{
  "commit_id": "{head_sha}",
  "event": "REQUEST_CHANGES",
  "body": "## ZéCode Review\n3 findings críticos...",
  "comments": [
    { "path": "src/api/bookings.ts", "line": 42,
      "body": "Missing input validation for `guestName`." }
  ]
}
```

2.4. Aba Actions (CI/CD)

A aba Actions é o motor de CI/CD nativo do GitHub. Workflows são definidos em YAML em .github/workflows/, disparados por eventos (push, pull_request, schedule, workflow_dispatch, repository_dispatch, webhook events). Jobs rodam em runners GitHub-hosted (Ubuntu, Windows, macOS) ou self-hosted. Suporta matrix builds, caching, services (Postgres, Redis, etc.), secrets (Settings → Secrets), environment protection rules, e approvals obrigatórios.

Como o ZéCode usa esta aba:

O ZéCode mantém um workflow principal `.github/workflows/ze-code-review.yml` que roda em todo PR aberto/sincronizado/reaberto. Este workflow: (1) faz checkout do código; (2) gera o diff entre base e head; (3) chama a API do ZCC `/api/zcc/ze-code/github-review` com o PR number e SHA; (4) faz poll até a review completar (timeout 5 min); (5) posta o resultado como comentário no PR usando o GITHUB_TOKEN nativo. Adicionalmente, o ZéCode pode ser acionado por `workflow_dispatch` (manual) e por `schedule` (cron diário para evolução autônoma em modo simulação).

2.5. Aba Projects (Gestão de Projetos)

A nova experiência Projects v2 é baseada em GraphQL, suporta views kanban/table/roadmap, campos customizados (status, iteration, single-select, number, date, text), filtros salvos, e agrupamentos. Issues e PRs podem ser vinculados a itens de projeto. É a ferramenta oficial de roadmap do GitHub, substituindo o Projects classic (que está sendo descontinuado).

Como o ZéCode usa esta aba:

O ZéCode cria items de projeto para cada `RefactorSuggestion` com prioridade alta, permitindo que a equipe visualize o roadmap de melhorias sugeridas em um único quadro. O campo "confidence" é mapeado para single-select (low/medium/high), "estimated_effort" para number (story points), e "created_at" para date. Via GraphQL mutation `addProjectV2ItemById`, o ZéCode adiciona a issue/PR ao projeto.

2.6. Aba Wiki (Documentação)

A Wiki é um subsistema Git separado que armazena páginas em Markdown com histórico versionado. Útil para documentação de longo prazo que não pertence ao repositório de código (runbooks, ADRs, arquitetura, guias de onboarding). Suporta sidebar e footer customizados, e é editável via Git clone da wiki (URL: `git@github.com:owner/repo.wiki.git`).

Como o ZéCode usa esta aba:

O ZéCode pode manter a wiki sincronizada com a documentação técnica do projeto. Quando detecta que uma `RefactorSuggestion` alterou a arquitetura (ex.: mudança em `schema.prisma`), ele atualiza a página de arquitetura na wiki via commit na branch main da wiki. Para isso usa Git Database API apontando para `{repo}.wiki.git`.

2.7. Aba Security (Segurança e Alertas)

A aba Security centraliza todas as funcionalidades de segurança do repositório: Dependabot alerts (vulnerabilidades em dependências), CodeQL analysis (SAST), Secret Scanning (detecta tokens vazados), Security policy (SECURITY.md), Advisory credits, Branch protection rules, e o Security overview (dashboard cross-repo). Esta aba é crítica para qualquer projeto production-grade.

Como o ZéCode usa esta aba:

O ZéCode consulta alertas de Dependabot via API para priorizar upgrades de dependências. Quando um CVE crítico é detectado, ele cria uma RefactorSuggestion de tipo "security-upgrade" com prioridade máxima, e abre um PR com o bump de versão (usando @dependabot como referência de padrão). O Secret Scanning é monitorado para detectar se algum PAT do ZéCode vazou acidentalmente em commits — se detectado, o Vault automaticamente revoga o PAT comprometido e notifica o admin via AlertBus.

```
// Listar alertas Dependabot
GET /repos/{owner}/{repo}/dependabot/alerts
  ?state=open&severity=critical&ecosystem=npm

// Verificar se houve push de secret
GET /repos/{owner}/{repo}/secret-scanning/alerts
  ?state=open
```

2.8. Aba Insights (Analytics e Métricas)

A aba Insights oferece 13 sub-views: Pulse, Contributors, Commit activity, Code frequency, Network, Forks, Repository traffic (clones/views), Dependents graph, Dependency graph, Stars history, Followers, Watchers. É o analytics nativo do GitHub — sem custo adicional, mas limitado a dados do próprio GitHub.

Como o ZéCode usa esta aba:

O ZéCode expõe um dashboard no ZCC que combina métricas do GitHub Insights com métricas internas (número de reviews, gaps detectados, PRs merged por ZéCode vs humanos). Isso permite responder perguntas como: "Qual a latência média entre gap detectado e PR merged?" ou "Quanto código foi refatorado pelo ZéCode este mês?". A API usada é /repos/{owner}/{repo}/stats/{type} (retorna dados agregados, pode demorar a gerar em repos grandes).

2.9. Aba Settings (Configurações)

A aba Settings é o painel administrativo do repositório. Sub-seções importantes: Options (rename, transfer, delete, default branch, merge strategy), Collaborators (per-user permissions), Branches (branch protection rules — crítica!), Tags, Webhooks (cadastro de webhook URLs), Secrets (Actions secrets), Variables (Actions variables), Deploy keys, Actions (permissions, runners), Environments (com protection rules e wait timers), Rulesets (nova API granular de regras).

Como o ZéCode usa esta aba:

O ZéCode NUNCA modifica Settings diretamente (princípio do menor privilégio). Em vez disso, ele recomenda configurações via issue com label "ze-code-recommendation". Exemplos: "Ativar branch protection em main: requires 2 reviews, requires status checks passing, requires linear history". O admin então aplica manualmente. A única exceção é Secrets — o ZéCode lê secrets (com audit log) mas nunca cria/atualiza/remove via API (apenas via UI para forçar auditoria humana).

PERIGO: Nunca configure Administration: write nem Webhooks: write no PAT do ZéCode. Esses escopos permitem criar webhooks maliciosos que exfiltram dados via push events, ou transferir ownership do repositório. Configure-os sempre via UI por um humano.

2.10. Aba Discussions (Fórum)

A aba Discussions é o fórum nativo do GitHub, alternativo ao GitHub Issues para perguntas, ideias, anúncios. Organizado por categorias (Announcements, Ideas, Q&A;, Show and tell), suporta threaded replies, markdown rico, e pode converter Discussão em Issue quando apropriado. Deve ser habilitado via Settings → Features → Discussions.

Como o ZéCode usa esta aba:

O ZéCode monitora Discussions para responder perguntas técnicas recorrentes automaticamente (ex.: "Como configurar webhook do ZéCode?" → responde com link para a wiki). Não posta proativamente — apenas responde a discussões marcadas com label "ze-code-help-needed". Isso reduz carga de suporte da equipe humana.

3. Comparativo de Métodos de Autenticação

A escolha do método de autenticação GitHub é a decisão de segurança mais importante desta integração. Esta seção compara em profundidade os cinco modelos disponíveis, com prós, contras, escopos exatos, e passo-a-passo de configuração.

3.1. PAT Clássico (NÃO RECOMENDADO)

O PAT clássico (Personal Access Token - Classic) é o modelo legado. É criado em Settings → Developer settings → Personal access tokens → Tokens (classic). Tem escopos amplos (repo, admin:org, etc.) que concedem acesso a TODOS os repositórios acessíveis pelo usuário. Sem expiração obrigatória (pode ser "no expiration"), sem restrição por repositório, sem auditoria granular. O GitHub recomenda fortemente migrar para Fine-Grained PATs e o PAT clássico está em desincentivo ativo (não recebe novos recursos).

NÃO USE. Pat clássico com escopo "repo" dá acesso a todos seus repositórios públicos E privados. Se vazado, compromete tudo. O ZéCode recusa esse tipo de token — uma validação no Vault detecta o prefixo "ghp_" e bloqueia com mensagem "Use Fine-Grained PAT (prefixo github_pat_)".

3.2. PAT Fine-Grained (Recomendado)

O Fine-Grained PAT é o modelo moderno e recomendado para uso pessoal ou de bots em pequena escala. É criado em Settings → Developer settings → Personal access tokens → Fine-grained tokens. Características:

Características principais:

- Prefixo: github_pat_ (diferente de ghp_ clássico)
- Escopo por repositório: seleciona individualmente quais repos o token acessa
- 40+ permissões granulares: Contents, Pull requests, Issues, Workflows, etc.
- Expiração obrigatória: máximo 1 ano, recomendado 90 dias
- Rate limit: 5000 req/hora por usuário
- Auditoria: visível em Settings → Security log

Passo a passo de configuração (execute antes de inserir no Vault):

1. Acesse: <https://github.com/settings/personal-access-tokens/new>
(Logado com conta corporativa nova, email @empresa)
2. Token name:
ze-code-prod-corporate
3. Expiration:
90 days (recomendado) – máximo aceitável: 1 ano
4. Resource owner:
[Sua organização corporativa ou usuário]
5. Repository access:
☒ Only select repositories
→ Selecione: SmartHotel_Zehla (e qualquer outro que o ZéCode operar)
6. Repository permissions (PERMISSÕES MÍNIMAS):
☐ Contents: Read and write
☐ Pull requests: Read and write
☐ Issues: Read and write
☐ Workflows: Read-only (NUNCA write)
☐ Metadata: Read-only (obrigatório, sempre incluído)

☐ Administration: NUNCA
☐ Webhooks: NUNCA
☐ Secrets: NUNCA
☐ Actions variables: NUNCA
☐ Deploy keys: NUNCA
☐ Environments: NUNCA
☐ Codespaces: NUNCA
☐ Commit statuses: opcional, Read-only se quiser status de CI
☐ Checks: opcional, Read-only se quiser status de checks
7. Generate token
→ COPIE o token imediatamente (github_pat_xxx...)
→ Ele NÃO aparece mais depois
8. Cole no Vault do ZCC:
ZCC → Settings → GitHub Credentials → Add
→ Label: ze-code-prod-corporate
→ Token: [cole aqui]
→ Click "Validate & Save"

Validação automática: Ao salvar o token, o Vault faz uma chamada GET /user para validar. Em seguida, faz GET /repos/{owner}/{repo} para cada repositório permitido e verifica que as permissões declaradas batem com as concedidas. Se divergir, o Vault rejeita o cadastro.

3.3. GitHub App (Gold Standard)

O GitHub App é o modelo gold standard para integrações SaaS e multi-organização. Em vez de um PAT atrelado a um usuário, você cria uma App (em Settings → Developer settings → GitHub Apps) que pode ser instalada em qualquer organização. A App se autentica via JWT (assinado com private key RSA) e troca o JWT por um token de instalação (expire em 1h, auto-renovável). Rate limit de 15000/h por instalação.

Vantagens sobre Fine-Grained PAT:

- Não atrelada a usuário: sobrevive a offboarding de funcionários
- Rate limit 3x maior: 15000/h vs 5000/h

- · Multi-org nativo: instala em múltiplas orgs sem novo token
- · Permissões por evento: granular via webhook subscriptions
- · Auto-rotação: JWT expira em 10 min, install token em 1h
- · Identidade própria: posts aparecem como "ZéCode Bot" em vez de seu usuário

Passo a passo de configuração:

1. Crie a GitHub App:
<https://github.com/settings/apps/new>
(Ou via organização: <https://github.com/organizations/{org}/settings/apps/new>)
2. GitHub App name:
ZéCode Bot (corporativo)
3. Homepage URL:
<https://zcc.zehla.local> (ou URL real do ZCC)
4. Webhook:
☒ Active
Webhook URL: <https://zcc.zehla.local/api/zcc/github/webhook>
Webhook secret: [gere um segredo forte de 32 chars]
5. Repository permissions:
☐ Contents: Read and write
☐ Pull requests: Read and write
☐ Issues: Read and write
☐ Workflows: Read-only
☐ Checks: Read-only
☐ Metadata: Read-only
6. Subscribe to events:
☒ Push
☒ Pull request
☒ Check suite
☒ Issue
☒ Check run
7. Where can this GitHub App be installed?
☒ Only on this account (mais seguro) ou Any account (se SaaS)
8. Create GitHub App
9. Após criar:
 - Anote o App ID (settings page)
 - Gere e baixe a Private Key (.pem file)
 - Anote o Client ID
10. Instale a App no repositório:
Settings → [Seu repo] → GitHub Apps → Install
11. No Vault do ZCC:
ZCC → Settings → GitHub Credentials → Add → Type: GitHub App
→ App ID: [número]
→ Client ID: [string]
→ Private Key: [cole o .pem completo]
→ Installation ID: [ache em /user/installations]

Trade-off: GitHub App é mais seguro mas exige manutenção da private key. Se você é uma operação pequena (1-3 repos, 1 organização), Fine-Grained PAT é suficiente. Se planeja escalar para multi-org ou SaaS, comece já com GitHub App.

3.4. Deploy Keys (Casos Específicos)

Deploy Keys são chaves SSH atreladas a UM repositório específico. São criadas em Settings → Deploy keys. Cada key pode ser read-only (padrão) ou read-write (marcando "Allow write access"). São ideais para CI/CD de servidores que precisam apenas clonar/push em um repo específico sem precisar de API access. Não suportam operações de PR, Issue, Actions — apenas Git push/pull/clone.

Para o ZéCode, Deploy Keys são insuficientes (ele precisa de API para PR/Issue/Actions). Mas são úteis para o sidecar de deploy do SmartHotel_Zehla, que precisa apenas fazer git pull no servidor de produção. Nesse caso, gere uma Deploy Key read-only específica para o servidor de produção.

3.5. OIDC (Para Actions)

OIDC (OpenID Connect) é o mecanismo pelo qual workflows do GitHub Actions autenticam em provedores cloud (AWS, GCP, Azure) sem precisar de secrets de longa duração. Cada job recebe um token JWT curto (10 min) assinado pelo GitHub, que o cloud valida. Elimina completamente o risco de vazamento de cloud credentials.

O ZéCode não usa OIDC diretamente (ele não é um workflow de Actions), mas os workflows que ele cria/mantém podem usar OIDC. Por exemplo, um workflow de deploy do SmartHotel_Zehla para VPS pode usar OIDC para autenticar no provedor cloud e baixar artefatos sem precisar de secrets estáticas.

PARTE II

Implementação

Esta parte documenta a implementação técnica completa, fase por fase. Cada fase é independente — pode ser implementada e testada isoladamente antes de avançar para a próxima. O código apresentado aqui está pronto para produção, com tratamento de erros, retries, rate limiting e audit logging.

4. Fase 1: PAT Vault Seguro

O PAT Vault é a camada crítica de segurança que armazena credenciais GitHub criptografadas em banco de dados. Esta fase é pré-requisito absoluto para todas as demais — sem ela, nenhuma outra fase pode operar com segurança.

4.1. Modelo de Dados (Prisma)

Duas novas tabelas são adicionadas ao schema do Prisma: `GitHubCredential` (armazena os PATs/Apps criptografados) e `PatAuditLog` (registro de toda leitura, uso, rotação e revogação). A separação em duas tabelas é deliberada: a tabela de audit log recebe dezenas de registros por hora (cada chamada de API é logada), enquanto a tabela de credenciais tem poucos registros mas altamente sensíveis. Isso permite otimizar índices e estratégias de backup separadamente.

```
// prisma/schema.prisma – ADICIONAR ao final

model GitHubCredential {
  id          String    @id @default(cuid())
  label       String    @unique // "prod-corporate" / "staging"
  authType    String    // "fine_grained_pat" | "github_app" |
  "deploy_key"
  encryptedPat String    @db.Text // AES-256-GCM ciphertext (PAT ou private key)
  patFingerprint String    // SHA-256 hash (busca rápida sem
  descriptografar)
  scopes      String[]   // permissões declaradas: ["contents:write",
  "pull_requests:write", ...]
  repositoryAccess String[] // repos permitidos:
  ["MarcioCau14/SmartHotel_Zehla", ...]
  githubAppId String?    // só para authType=github_app
  githubClientId String? // só para authType=github_app
  installationId String? // só para authType=github_app
  webhookSecret String?  // criptografado, só para github_app
  expiresAt    DateTime
  rotatedFromId String?
  rotatedFrom  GitHubCredential? @relation("RotationChain", fields:
  [rotatedFromId], references: [id])
  rotatedTo    GitHubCredential[]
  createdBy    String      // admin user id (NextAuth)
  isActive     Boolean    @default(true)
  lastUsedAt   DateTime?
  lastValidatedAt DateTime?
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
}
```

```

auditLogs      PatAuditLog[]

@@index([patFingerprint])
@@index([isActive])
@@index([expiresAt])
}

model PatAuditLog {
  id            String   @id @default(cuid())
  credentialId  String
  credential    GitHubCredential @relation(fields: [credentialId], references: [id])
  action        String   // "VALIDATE" | "READ" | "ROTATE" | "REVOKE" | "API_CALL" |
"EXPIRE_ALERT"
  apiEndpoint   String?  // ex: "POST /repos/MarcioCau14/SmartHotel_Zehla/pulls"
  apiMethod     String?  // "GET" | "POST" | ...
  repository    String?
  statusCode    Int?
  success       Boolean
  errorMessage  String?  @db.Text
  ipAddress     String?
  userAgent     String?
  durationMs    Int?
  createdAt     DateTime @default(now())

  @@index([credentialId, createdAt])
  @@index([action, createdAt])
  @@index([success, createdAt])
}

```

4.2. Camada de Criptografia (AES-256-GCM)

A criptografia usa AES-256-GCM, que além de confidencialidade (ninguém sem a key pode ler) oferece autenticação (auth tag detecta se o ciphertext foi adulterado). A master key é lida de variável de ambiente GITHUB_PAT_MASTER_KEY — deve ter 32 bytes (256 bits), codificada em base64. Essa key NUNCA entra no banco de dados nem no código; é provisionada pelo servidor ou KMS externo (AWS KMS, GCP KMS, HashiCorp Vault).

```

// src/lib/github/pat-vault.ts

import crypto from 'crypto';
import { prisma } from '@lib/db/tenant-prisma';

const MASTER_KEY_B64 = process.env.GITHUB_PAT_MASTER_KEY;
if (!MASTER_KEY_B64) {
  throw new Error('GITHUB_PAT_MASTER_KEY env var is required (32 bytes base64)');
}
const MASTER_KEY = Buffer.from(MASTER_KEY_B64, 'base64');
if (MASTER_KEY.length !== 32) {
  throw new Error(`Master key must be 32 bytes, got ${MASTER_KEY.length}`);
}

const ALGO = 'aes-256-gcm';
const IV_LENGTH = 12; // GCM standard
const TAG_LENGTH = 16;

export interface EncryptedPayload {
  iv: string; // base64
  ciphertext: string; // base64
  tag: string; // base64
}

export function encryptPat(plaintext: string): EncryptedPayload {

```

```
const iv = crypto.randomBytes(IV_LENGTH);
const cipher = crypto.createCipheriv(ALGO, MASTER_KEY, iv);
const ciphertext = Buffer.concat([
  cipher.update(plaintext, 'utf8'),
  cipher.final()
]);
const tag = cipher.getAuthTag();
return {
  iv: iv.toString('base64'),
  ciphertext: ciphertext.toString('base64'),
  tag: tag.toString('base64')
};
}

export function decryptPat(payload: EncryptedPayload): string {
  const iv = Buffer.from(payload.iv, 'base64');
  const tag = Buffer.from(payload.tag, 'base64');
  const ciphertext = Buffer.from(payload.ciphertext, 'base64');
  const decipher = crypto.createDecipheriv(ALGO, MASTER_KEY, iv);
  decipher.setAuthTag(tag);
  const plaintext = Buffer.concat([
    decipher.update(ciphertext),
    decipher.final()
  ]).toString('utf8');
  return plaintext;
}

export function fingerprintPat(pat: string): string {
  return crypto.createHash('sha256').update(pat).digest('hex');
}

// Persiste nova credencial (com validação)
export async function createCredential(opts: {
  label: string;
  authType: 'fine_grained_pat' | 'github_app' | 'deploy_key';
  pat: string; // PAT ou private key PEM
  scopes: string[];
  repositoryAccess: string[];
  expiresAt: Date;
  createdBy: string;
  githubAppId?: string;
  githubClientId?: string;
  installationId?: string;
  webhookSecret?: string;
}): Promise<string> {
  // 1. Dedup por fingerprint – não permite cadastrar mesmo PAT duas vezes
  const fp = fingerprintPat(opts.pat);
  const existing = await prisma.gitHubCredential.findFirst({
    where: { patFingerprint: fp, isActive: true }
  });
  if (existing) {
    throw new Error(`PAT já cadastrado como "${existing.label}". Reuse ou revogue antes.`);
  }

  // 2. Criptografa
  const enc = encryptPat(opts.pat);
  const encStr = JSON.stringify(enc);

  // 3. Persiste
  const cred = await prisma.gitHubCredential.create({
    data: {
      label: opts.label,
      authType: opts.authType,
      encryptedPat: encStr,
```

```

        patFingerprint: fp,
        scopes: opts.scopes,
        repositoryAccess: opts.repositoryAccess,
        expiresAt: opts.expiresAt,
        createdBy: opts.createdBy,
        githubAppId: opts.githubAppId,
        githubClientId: opts.githubClientId,
        installationId: opts.installationId,
        webhookSecret: opts.webhookSecret ?
JSON.stringify(encryptPat(opts.webhookSecret)) : null,
    }
  });

  // 4. Audit
  await auditLog(cred.id, 'VALIDATE', { success: true, action: 'CREATE' });

  return cred.id;
}

export async function getCredential(credentialId: string): Promise<string> {
  const cred = await prisma.gitHubCredential.findUniqueOrThrow({
    where: { id: credentialId }
  });
  if (!cred.isActive) throw new Error('Credential revoked or inactive');
  if (cred.expiresAt < new Date()) throw new Error('Credential expired');

  const payload = JSON.parse(cred.encryptedPat) as EncryptedPayload;
  const pat = decryptPat(payload);

  // Atualiza lastUsedAt (não aguarda para não bloquear)
  prisma.gitHubCredential.update({
    where: { id: credentialId },
    data: { lastUsedAt: new Date() }
  }).catch(() => {}); // silent fail on audit update

  return pat;
}

export async function auditLog(
  credentialId: string,
  action: string,
  details: {
    apiEndpoint?: string;
    apiMethod?: string;
    repository?: string;
    statusCode?: number;
    success: boolean;
    errorMessage?: string;
    ipAddress?: string;
    userAgent?: string;
    durationMs?: number;
  }
) {
  await prisma.patAuditLog.create({
    data: {
      credentialId,
      action,
      ...details
    }
  }).catch(() => {}); // silent fail – audit nunca bloqueia operação
}

// Rotação zero-downtime
export async function rotateCredential(
  oldId: string,

```

```
newPat: string,
newExpiresAt: Date,
actorUserId: string
): Promise<string> {
  const old = await prisma.gitHubCredential.findUniqueOrThrow({ where: { id: oldId }
});

  // 1. Valida novo PAT
  const fp = fingerprintPat(newPat);
  const enc = encryptPat(newPat);

  // 2. Cria novo registro (mesma label + "-rotated-{timestamp}")
  const newCred = await prisma.gitHubCredential.create({
    data: {
      label: `${old.label}-rotated-${Date.now()}`,
      authType: old.authType,
      encryptedPat: JSON.stringify(enc),
      patFingerprint: fp,
      scopes: old.scopes,
      repositoryAccess: old.repositoryAccess,
      expiresAt: newExpiresAt,
      rotatedFromId: oldId,
      createdBy: actorUserId,
      githubAppId: old.githubAppId,
      githubClientId: old.githubClientId,
      installationId: old.installationId,
    }
  });

  // 3. Marca antigo como inativo (não deleta – mantém histórico)
  await prisma.gitHubCredential.update({
    where: { id: oldId },
    data: { isActive: false }
  });

  // 4. Tenta revogar antigo no GitHub (PAT Fine-Grained: DELETE /applications/token)
  // Isso só funciona se o cliente souber o Client ID/Secret da App que gerou o PAT.
  // Para PATs gerados via UI, revogação é manual via Settings.

  // 5. Audit
  await auditLog(oldId, 'ROTATE', { success: true, action: 'ROTATED_TO:' + newCred.id
});
  await auditLog(newCred.id, 'ROTATE', { success: true, action: 'ROTATED_FROM:' + oldId
});

  return newCred.id;
}
```

4.3. Variáveis de Ambiente Necessárias

```
# .env (NUNCA commitar)
# Gerar com: openssl rand -base64 32
GITHUB_PAT_MASTER_KEY="sua_chave_de_32_bytes_em_base64_aqui"

# URL pública do ZCC (para webhook do GitHub App)
ZCC_PUBLIC_URL="https://zcc.seuzella.com"

# Token secreto para o GitHub Action chamar a API do ZCC
# Gerar com: openssl rand -hex 32
ZE_CODE_WEBHOOK_TOKEN="seu_token_secreto_aqui"

# Para GitHub App (opcional, se usar):
GITHUB_APP_APP_ID=""
GITHUB_APP_CLIENT_ID=""
GITHUB_APP_PRIVATE_KEY_PATH="./keys/ze-code-bot.private-key.pem"
GITHUB_APP_INSTALLATION_ID=""
GITHUB_APP_WEBHOOK_SECRET=""
```

5. Fase 2: GitHub Client Service

O GitHubClient é o wrapper centralizado para todas as chamadas à API GitHub. Toda operação GitHub (seja do Evolve-to-PR, do Webhook Listener, ou do GitHub Action) passa por ele. Isso garante rate limiting consistente, retries com backoff, audit log de cada chamada, e validação de escopo antes de disparar a request.

5.1. Estrutura Principal

```
// src/lib/github/github-client.ts

import { getCredential, auditLog } from './pat-vault';

const GITHUB_API = 'https://api.github.com';
const USER_AGENT = 'ZéCode-Bot/1.0 (+https://zcc.seuzella.com)';

interface RequestOptions {
  method: 'GET' | 'POST' | 'PATCH' | 'PUT' | 'DELETE';
  path: string; // ex: /repos/owner/repo/pulls
  body?: any;
  headers?: Record<string, string>;
  expectedStatus?: number[];
}

interface RateLimitInfo {
  limit: number;
  remaining: number;
  reset: Date;
  used: number;
}

export class GitHubClient {
  private rateLimit: RateLimitInfo | null = null;
  private etagCache = new Map<string, { etag: string; data: any }>();

  constructor(private credentialId: string) {}

  private async getAuthHeader(): Promise<string> {
    const cred = await prisma.gitHubCredential.findUniqueOrThrow({
      where: { id: this.credentialId }
    });

    if (cred.authType === 'fine_grained_pat') {
      const pat = await getCredential(this.credentialId);
      return `Bearer ${pat}`;
    }

    if (cred.authType === 'github_app') {
      const token = await getInstallationToken(cred);
      return `token ${token}`;
    }

    throw new Error(`Unsupported authType: ${cred.authType}`);
  }

  private async callApi(opts: RequestOptions): Promise<any> {
    const startTime = Date.now();
    const authHeader = await this.getAuthHeader();

    // Rate limit check – se resetar, espera até reset
    if (this.rateLimit && this.rateLimit.remaining === 0) {
      const waitMs = this.rateLimit.reset.getTime()() - Date.now();
      if (waitMs > 0 && waitMs < 60000) {
        await new Promise(r => setTimeout(r, waitMs + 100));
      }
    }

    const url = `${GITHUB_API}${opts.path}`;
    const headers: Record<string, string> = {
      'Authorization': authHeader,
      'Accept': 'application/vnd.github+json',
      'X-GitHub-Api-Version': '2022-11-28',
    };
  }
}
```

```
'User-Agent': USER_AGENT,
...opts.headers
};

// ETag caching para GETs
if (opts.method === 'GET') {
  const cached = this.etagCache.get(opts.path);
  if (cached) headers['If-None-Match'] = cached.etag;
}

// Loga pré-call
const auditId = auditLog(this.credentialId, 'API_CALL', {
  apiEndpoint: opts.path,
  apiMethod: opts.method,
  success: true // assume sucesso, atualiza depois
});

let attempt = 0;
const maxRetries = 3;

while (attempt <= maxRetries) {
  attempt++;
  try {
    const res = await fetch(url, {
      method: opts.method,
      headers,
      body: opts.body ? JSON.stringify(opts.body) : undefined
    });

    // Atualiza rate limit info
    this.updateRateLimit(res.headers);

    // 304 Not Modified – usa cache
    if (res.status === 304 && this.etagCache.has(opts.path)) {
      return this.etagCache.get(opts.path)!.data;
    }

    // Retry em 429 (rate limit), 502, 503, 504
    if ([429, 502, 503, 504].includes(res.status) && attempt <= maxRetries) {
      const backoffMs = Math.min(1000 * 2 ** attempt, 30000);
      await new Promise(r => setTimeout(r, backoffMs));
      continue;
    }

    // 404 – normal para alguns endpoints
    if (res.status === 404) {
      await auditLog(this.credentialId, 'API_CALL', {
        apiEndpoint: opts.path, apiMethod: opts.method,
        success: false, statusCode: 404,
        errorMessage: 'Not Found', durationMs: Date.now() - startTime
      });
      return null;
    }

    if (!res.ok && !(opts.expectedStatus || []).includes(res.status)) {
      const errBody = await res.text();
      await auditLog(this.credentialId, 'API_CALL', {
        apiEndpoint: opts.path, apiMethod: opts.method,
        success: false, statusCode: res.status,
        errorMessage: errBody, durationMs: Date.now() - startTime
      });
      throw new GitHubApiError(res.status, errBody, opts.path);
    }

    const data = res.status === 204 ? null : await res.json();
```

```

// Atualiza cache ETag
if (opts.method === 'GET' && res.headers.get('ETag')) {
  this.etagCache.set(opts.path, {
    etag: res.headers.get('ETag')!,
    data
  });
}

await auditLog(this.credentialId, 'API_CALL', {
  apiEndpoint: opts.path, apiMethod: opts.method,
  success: true, statusCode: res.status,
  durationMs: Date.now() - startTime
});

return data;

} catch (err) {
  if (attempt > maxRetries) {
    await auditLog(this.credentialId, 'API_CALL', {
      apiEndpoint: opts.path, apiMethod: opts.method,
      success: false, errorMessage: String(err),
      durationMs: Date.now() - startTime
    });
    throw err;
  }
  await new Promise(r => setTimeout(r, 1000 * 2 ** attempt));
}
}

private updateRateLimit(headers: Headers) {
  const remaining = headers.get('X-RateLimit-Remaining');
  const reset = headers.get('X-RateLimit-Reset');
  const limit = headers.get('X-RateLimit-Limit');
  const used = headers.get('X-RateLimit-Used');
  if (remaining && reset) {
    this.rateLimit = {
      remaining: parseInt(remaining),
      reset: new Date(parseInt(reset) * 1000),
      limit: limit ? parseInt(limit) : 5000,
      used: used ? parseInt(used) : 0
    };
  }
}

// ===== BRANCHES & COMMITS =====

async createBranch(repo: string, fromBranch: string, newBranch: string) {
  const ref = await this.callApi({
    method: 'GET',
    path: `/repos/${repo}/git/refs/heads/${fromBranch}`
  });
  return this.callApi({
    method: 'POST',
    path: `/repos/${repo}/git/refs`,
    body: { ref: `refs/heads/${newBranch}`, sha: ref.object.sha }
  });
}

async commitFiles(repo: string, branch: string, files: Array<{
  path: string; content: string; encoding?: 'utf-8' | 'base64'
}>, message: string): Promise<{ sha: string; commitUrl: string }> {
  // 1. Pega a árvore atual do branch
  const ref = await this.callApi({

```

```
    method: 'GET', path: `/repos/${repo}/git/refs/heads/${branch}`
  });
  const baseCommit = await this.callApi({
    method: 'GET', path: `/repos/${repo}/git/commits/${ref.object.sha}`
  });

  // 2. Cria blobs para cada arquivo
  const treeItems = [];
  for (const file of files) {
    const blob = await this.callApi({
      method: 'POST',
      path: `/repos/${repo}/git/blobs`,
      body: {
        content: file.content,
        encoding: file.encoding || 'utf-8'
      }
    });
    treeItems.push({
      path: file.path,
      mode: '100644',
      type: 'blob',
      sha: blob.sha
    });
  }

  // 3. Cria nova árvore baseada na anterior
  const tree = await this.callApi({
    method: 'POST',
    path: `/repos/${repo}/git/trees`,
    body: { base_tree: baseCommit.tree.sha, tree: treeItems }
  });

  // 4. Cria commit apontando para nova árvore
  const commit = await this.callApi({
    method: 'POST',
    path: `/repos/${repo}/git/commits`,
    body: {
      message,
      tree: tree.sha,
      parents: [baseCommit.sha],
      author: {
        name: 'ZéCode Bot',
        email: 'ze-code@zehla.local',
        date: new Date().toISOString()
      }
    }
  });

  // 5. Atualiza o branch para apontar para o novo commit
  await this.callApi({
    method: 'PATCH',
    path: `/repos/${repo}/git/refs/heads/${branch}`,
    body: { sha: commit.sha }
  });

  return { sha: commit.sha, commitUrl: commit.html_url };
}

// ===== PULL REQUESTS =====

async createPR(opts: {
  repo: string; title: string; head: string; base: string;
  body: string; draft?: boolean;
}) {
  return this.callApi({
```

```

        method: 'POST',
        path: `/repos/${opts.repo}/pulls`,
        body: {
            title: opts.title, head: opts.head, base: opts.base,
            body: opts.body, draft: opts.draft || false
        }
    });
}

async addPRComment(repo: string, prNumber: number, body: string) {
    return this.callApi({
        method: 'POST',
        path: `/repos/${repo}/issues/${prNumber}/comments`,
        body: { body }
    });
}

async createPRReview(repo: string, prNumber: number, review: {
    commitId: string;
    event: 'APPROVE' | 'REQUEST_CHANGES' | 'COMMENT';
    body: string;
    comments?: Array<{ path: string; line: number; body: string }>;
}) {
    return this.callApi({
        method: 'POST',
        path: `/repos/${repo}/pulls/${prNumber}/reviews`,
        body: review
    });
}

async getPRDiff(repo: string, prNumber: number): Promise<string> {
    return this.callApi({
        method: 'GET',
        path: `/repos/${repo}/pulls/${prNumber}`,
        headers: { 'Accept': 'application/vnd.github.v3.diff' }
    });
}

async mergePR(repo: string, prNumber: number, method: 'merge' | 'squash' | 'rebase')
{
    return this.callApi({
        method: 'PUT',
        path: `/repos/${repo}/pulls/${prNumber}/merge`,
        body: { merge_method: method }
    });
}

// ===== ISSUES =====

async createIssue(repo: string, title: string, body: string, labels?: string[]) {
    return this.callApi({
        method: 'POST',
        path: `/repos/${repo}/issues`,
        body: { title, body, labels: labels || [] }
    });
}

// ===== CI/CD =====

async getWorkflowRuns(repo: string, branch?: string) {
    const params = new URLSearchParams({ per_page: '20' });
    if (branch) params.set('branch', branch);
    return this.callApi({
        method: 'GET',
        path: `/repos/${repo}/actions/runs?${params}`
    });
}

```

```

    });
  }

  async getCheckStatuses(repo: string, ref: string) {
    return this.callApi({
      method: 'GET',
      path: `/repos/${repo}/commits/${ref}/check-runs`
    });
  }

  // ===== VALIDATION =====

  async validatePat(pat: string): Promise<{
    valid: boolean; login: string; scopes: string[];
    rateLimit: { remaining: number; reset: Date };
  }> {
    const res = await fetch(`${GITHUB_API}/user`, {
      headers: {
        'Authorization': `Bearer ${pat}`,
        'Accept': 'application/vnd.github+json',
        'User-Agent': USER_AGENT
      }
    });
    if (!res.ok) return { valid: false, login: '', scopes: [], rateLimit: { remaining:
0, reset: new Date() } };
    const data = await res.json();
    const scopesHeader = res.headers.get('X-0Auth-Scopes') || '';
    return {
      valid: true,
      login: data.login,
      scopes: scopesHeader.split(',').map(s => s.trim()).filter(Boolean),
      rateLimit: {
        remaining: parseInt(res.headers.get('X-RateLimit-Remaining') || '0'),
        reset: new Date(parseInt(res.headers.get('X-RateLimit-Reset') || '0') * 1000)
      }
    };
  }
}

class GitHubApiError extends Error {
  constructor(public status: number, public body: string, public path: string) {
    super(`GitHub API ${status} on ${path}: ${body}`);
  }
}

// Singleton cache por credentialId
const clientCache = new Map<string, GitHubClient>();
export function getGitHubClient(credentialId: string): GitHubClient {
  if (!clientCache.has(credentialId)) {
    clientCache.set(credentialId, new GitHubClient(credentialId));
  }
  return clientCache.get(credentialId)!;
}

```

6. Fase 3: Evolve-to-PR Pipeline

O Evolve-to-PR é o pipeline que transforma uma RefactorSuggestion detectada pelo ZéCode em um Pull Request aberto no GitHub. Este pipeline é o coração operacional do ZéCode — é onde a inteligência de código (gap-detector + bottleneck-detector) encontra a ação concreta (commit + PR).

6.1. Fluxo Completo



6.2. Implementação do Service

```
// src/lib/cerebro/ze-code/apply-via-pr.ts

import { getGitHubClient } from '@lib/github/github-client';
import { prisma } from '@lib/db/tenant-prisma';
import { auditLog } from '@lib/github/pat-vault';

const MAX_OPEN_PRS = 3; // limite de PRs abertos pelo ZéCode simultaneamente
const BRANCH_PREFIX = 'feat/ze-code/';

export interface ApplyViaPROpts {
  suggestionId: string;
  credentialId: string;
  dryRun?: boolean; // se true, só simula – não cria PR
  createIssue?: boolean; // se true, cria issue além do PR
  actorUserId: string;
}

export async function applySuggestionViaPR(opts: ApplyViaPROpts) {
  // 1. Carrega suggestion
  const suggestion = await prisma.refactorSuggestion.findUniqueOrThrow({
    where: { id: opts.suggestionId },
    include: { review: true }
  });

  if (suggestion.status !== 'pending') {
    throw new Error(`Suggestion must be pending, got ${suggestion.status}`);
  }

  // 2. Verifica limite de PRs abertos
  const openPRs = await prisma.refactorSuggestion.count({
    where: { status: 'pr_open', createdBy: 'ze-code-bot' }
  });
  if (openPRs >= MAX_OPEN_PRS) {
    throw new Error(`Limite de ${MAX_OPEN_PRS} PRs abertos pelo ZéCode atingido.
    Aguarde merge de algum.`);
  }

  // 3. Pega credencial
  const client = getGitHubClient(opts.credentialId);

  // 4. Determina repositório (assume single-repo por enquanto)
  const repo = 'MarcioCaul4/SmartHotel_Zehla';
  const baseBranch = 'main';
  const branchName = `${BRANCH_PREFIX}${suggestion.slug}-${Date.now().toString(36)}`;

  // 5. Dry-run mode: só retorna o que faria
  if (opts.dryRun) {
    return {
      dryRun: true,
      branchName,
      files: suggestion.patch.files.map(f => ({ path: f.path, additions: f.additions,
      deletions: f.deletions })),
      commitMessage: `[ze-code-automated] ${suggestion.title}`,
      prTitle: `[ze-code-automated] ${suggestion.title}`,
      prBody: renderPRBody(suggestion)
    };
  }

  // 6. Cria branch
  await client.createBranch(repo, baseBranch, branchName);

  // 7. Commit atômico (múltiplos arquivos)
  const commitMessage = `[ze-code-automated] ${suggestion.title}`
}
```

```

Suggestion ID: ${suggestion.id}
Confidence: ${suggestion.confidence * 100}.toFixed(1)}%
Detected by: ${suggestion.detectedBy}
Gap type: ${suggestion.gapType}

${suggestion.description}

Co-authored-by: ZéCode <ze-code@zehla.local>`;

const commitResult = await client.commitFiles(
  repo, branchName,
  suggestion.patch.files.map(f => ({ path: f.path, content: f.newContent })),
  commitMessage
);

// 8. Abre PR
const prBody = renderPRBody(suggestion);
const pr = await client.createPR({
  repo, title: `[ze-code-automated] ${suggestion.title}`,
  head: branchName, base: baseBranch, body: prBody, draft: false
});

// 9. Cria issue linked (opcional)
let issueNumber: number | null = null;
if (opts.createIssue) {
  const issue = await client.createIssue(
    repo,
    `[ze-code-automated] ${suggestion.title}`,
    `## Contexto\n\nSugerido por ZéCode em ${suggestion.createdAt.toISOString()}\n\n`
+
    `Linked to PR #${pr.number}\n\n${suggestion.description}`,
    ['ze-code-automated', `severity:${suggestion.severity}`]
  );
  issueNumber = issue.number;
}

// 10. Atualiza suggestion
await prisma.refactorSuggestion.update({
  where: { id: suggestion.id },
  data: {
    status: 'pr_open',
    prUrl: pr.html_url,
    prNumber: pr.number,
    issueNumber,
    branchName,
    commitSha: commitResult.sha,
    appliedBy: opts.actorUserId,
    appliedAt: new Date()
  }
});

// 11. Audit
await auditLog(opts.credentialId, 'API_CALL', {
  apiEndpoint: `POST /repos/${repo}/pulls`,
  apiMethod: 'POST',
  repository: repo,
  success: true,
  statusCode: 201
});

return {
  success: true,
  prUrl: pr.html_url,
  prNumber: pr.number,

```

```
    issueNumber,
    branchName,
    commitSha: commitResult.sha
  };
}

function renderPRBody(suggestion: any): string {
  return `## 🔄 ZéCode Automated Refactor

### Justificativa
${suggestion.description}

### Detecção
- **Tipo de gap:** \`${suggestion.gapType}\`
- **Heurística:** \`${suggestion.detectedBy}\`
- **Confiança:** ${suggestion.confidence * 100}.toFixed(1)%
- **Severidade:** \`${suggestion.severity}\`

### Arquivos alterados
${suggestion.patch.files.map((f: any) =>
  `- \`${f.path}\` (+${f.additions} -${f.deletions})`
).join('\n')}

### Checklist de revisão
- [ ] A mudança resolve o gap identificado?
- [ ] Não introduz regressão em testes?
- [ ] Mantém ou melhora a legibilidade?
- [ ] Não quebra compatibilidade de API pública?

### Metadados
- Suggestion ID: \`${suggestion.id}\`
- Generated at: ${new Date().toISOString()}

---
*Este PR foi criado automaticamente pelo ZéCode. Revise com atenção e merge apenas se
concordar com a mudança.*`
}
```

7. Fase 4: GitHub Action de PR Review

O GitHub Action é o mecanismo que faz o ZéCode reagir automaticamente a todo PR aberto no repositório. Ele é implementado como um workflow YAML em `.github/workflows/ze-code-review.yml`, dispara em eventos `pull_request` (`opened/synchronize/reopened`), e orquestra a chamada à API do ZCC para análise.

7.1. Workflow YAML

```
# .github/workflows/ze-code-review.yml
name: ZéCode PR Review

on:
  pull_request:
    types: [opened, synchronize, reopened]
    # Não rodar em draft PRs (economiza recursos)
    # Descomente para filtrar paths:
    # paths:
    #   - 'src/**'
    #   - 'prisma/**'
    #   - 'package.json'

permissions:
  contents: read
  pull-requests: write
  checks: write
  # NUNCA conceder: administrations, secrets, workflows (write)

jobs:
  ze-code-review:
    runs-on: ubuntu-latest
    timeout-minutes: 10

    steps:
      - name: Checkout code
        uses: actions/checkout@v4
        with:
          fetch-depth: 0 # necessário para diff completo

      - name: Check if ZéCode-automated PR (skip to avoid loop)
        id: check
        env:
          GH_TOKEN: ${ secrets.GITHUB_TOKEN }
        run: |
          PR_AUTHOR=$(gh pr view ${ github.event.pull_request.number } \
            --json author --jq '.author.login')

          # Pula se for PR criado pelo ZéCode (evita loop infinito)
          if [[ "$PR_AUTHOR" == "ze-code-bot[bot]" ]] || \
            [[ "$PR_AUTHOR" == *"ze-code-automated"* ]]; then
            echo "skip=true" >> $GITHUB_OUTPUT
            echo "ZéCode-automated PR detected, skipping review."
          else
            echo "skip=false" >> $GITHUB_OUTPUT
          fi

      - name: Get PR diff size
        if: steps.check.outputs.skip != 'true'
        id: diff
        run: |
          git diff origin/${ github.base_ref }...HEAD > /tmp/pr.diff
          SIZE=$(wc -c < /tmp/pr.diff)
          echo "size=$SIZE" >> $GITHUB_OUTPUT
          echo "Diff size: $SIZE bytes"

      - name: Skip if too large (>500KB)
        if: |
          steps.check.outputs.skip != 'true' &&
          steps.diff.outputs.size > 500000
        run: |
          echo "::warning::Diff too large (${ steps.diff.outputs.size } bytes),
          skipping ZéCode review"
```

```

- name: Call ZéCode API
  if: |
    steps.check.outputs.skip != 'true' &&
    steps.diff.outputs.size <= 500000
  id: review
  env:
    ZE_CODE_API_URL: ${ secrets.ZE_CODE_API_URL }
    ZE_CODE_WEBHOOK_TOKEN: ${ secrets.ZE_CODE_WEBHOOK_TOKEN }
  run: |
    RESPONSE=$(curl -s -X POST \
      -H "Authorization: Bearer $ZE_CODE_WEBHOOK_TOKEN" \
      -H "Content-Type: application/json" \
      -d "{
        \"repository\": \"${ github.repository }\",
        \"prNumber\": ${ github.event.pull_request.number },
        \"headSha\": \"${ github.event.pull_request.head.sha }\",
        \"baseRef\": \"${ github.base_ref }\",
        \"headRef\": \"${ github.event.pull_request.head.ref }\",
        \"diffUrl\": \"${ github.event.pull_request.diff_url }\",
        \"author\": \"${ github.event.pull_request.user.login }\"
      }" \
      "$ZE_CODE_API_URL/api/zcc/ze-code/github-review")

    REVIEW_ID=$(echo "$RESPONSE" | jq -r '.reviewId // empty')

    if [ -z "$REVIEW_ID" ]; then
      echo "::error::Failed to start ZéCode review: $RESPONSE"
      exit 1
    fi

    echo "review_id=$REVIEW_ID" >> $GITHUB_OUTPUT
    echo "Review started: $REVIEW_ID"

- name: Poll for review completion
  if: steps.review.outputs.review_id
  env:
    ZE_CODE_API_URL: ${ secrets.ZE_CODE_API_URL }
    ZE_CODE_WEBHOOK_TOKEN: ${ secrets.ZE_CODE_WEBHOOK_TOKEN }
  run: |
    REVIEW_ID="${ steps.review.outputs.review_id }"
    MAX_POLLS=30
    POLL_INTERVAL=10 # segundos

    for i in $(seq 1 $MAX_POLLS); do
      STATUS=$(curl -s \
        -H "Authorization: Bearer $ZE_CODE_WEBHOOK_TOKEN" \
        "$ZE_CODE_API_URL/api/zcc/ze-code/reviews/$REVIEW_ID/status" \
        | jq -r '.status')

      echo "Poll $i/$MAX_POLLS: status=$STATUS"

      if [ "$STATUS" = "completed" ]; then
        echo "Review completed!"
        exit 0
      fi

      if [ "$STATUS" = "failed" ] || [ "$STATUS" = "error" ]; then
        echo "::error::ZéCode review failed"
        exit 1
      fi

      sleep $POLL_INTERVAL
    done

```

```
echo "::warning::ZéCode review timed out after 5 minutes"
exit 0
```

7.2. Secrets Necessários no GitHub

Configure em Settings → Secrets and variables → Actions → New repository secret:

Secret Name	Valor	Onde Obter
ZE_CODE_API_URL	https://zcc.seuzella.com	URL pública do ZCC (sem barra final)
ZE_CODE_WEBHOOK_TOKEN	[string aleatória de 64 chars]	openssl rand -hex 32

O GITHUB_TOKEN é automático (não precisa configurar). Ele tem escopos limitados definidos em permissions: no workflow acima, só contents:read e pull-requests:write. Isso garante que mesmo que o workflow seja comprometido, não pode fazer push direto, alterar settings, etc.

8. Fase 5: Webhook Listener

O Webhook Listener é o endpoint `/api/zcc/github/webhook` que recebe eventos do GitHub em tempo real. Para cada evento relevante (`push`, `pull_request`, `check_suite`, `issue`), o listener dispara ações no ZéCode: reindexar código, atualizar `RefactorSuggestions`, sincronizar issues, etc.

8.1. Endpoint de Webhook

```
// src/app/api/zcc/github/webhook/route.ts

import { NextRequest, NextResponse } from 'next/server';
import crypto from 'crypto';

const GITHUB_IP_RANGES = [
  '192.30.252.0/22',
  '185.199.108.0/22',
  '140.82.112.0/16',
  '143.55.64.0/20',
  '2a0a:a440::/29',
  '2606:50c0:8000::/24'
];

function isGitHubIP(ip: string): boolean {
  for (const range of GITHUB_IP_RANGES) {
    // Implementação simplificada – use ip-cidr em produção
    if (isIPInCIDR(ip, range)) return true;
  }
  return false;
}

function verifyWebhookSignature(payload: string, signature: string, secret: string):
boolean {
  const expected = 'sha256=' + crypto
    .createHmac('sha256', secret)
    .update(payload)
    .digest('hex');
  // Constant-time comparison
  return crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(signature));
}

export async function POST(req: NextRequest) {
  const signature = req.headers.get('X-Hub-Signature-256') || '';
  const eventType = req.headers.get('X-GitHub-Event') || '';
  const deliveryId = req.headers.get('X-GitHub-Delivery') || '';
  const clientIP = req.headers.get('x-forwarded-for')?.split(',')[0] || '';

  // 1. IP allowlist (defesa em profundidade – mesmo com signature válida)
  if (process.env.NODE_ENV === 'production' && !isGitHubIP(clientIP)) {
    return NextResponse.json({ error: 'Forbidden IP' }, { status: 403 });
  }

  const body = await req.text();
  const secret = process.env.GITHUB_APP_WEBHOOK_SECRET ||
process.env.GITHUB_WEBHOOK_SECRET;
  if (!secret) {
    console.error('GITHUB_WEBHOOK_SECRET not configured');
    return NextResponse.json({ error: 'Server misconfigured' }, { status: 500 });
  }

  // 2. Verifica assinatura HMAC
  if (!verifyWebhookSignature(body, signature, secret)) {
    console.warn(`Webhook signature mismatch for delivery ${deliveryId}`);
    return NextResponse.json({ error: 'Invalid signature' }, { status: 401 });
  }

  // 3. Idempotência via X-GitHub-Delivery
  // (poderia checar em Redis/cache se já processamos esse deliveryId)

  const payload = JSON.parse(body);
  console.log(`Webhook ${eventType} [${deliveryId}] from
${payload.repository?.full_name}`);
}
```

```
// 4. Dispatch por tipo de evento
try {
  switch (eventType) {
    case 'push':
      await handlePush(payload);
      break;
    case 'pull_request':
      await handlePullRequest(payload);
      break;
    case 'check_suite':
      await handleCheckSuite(payload);
      break;
    case 'issues':
      await handleIssue(payload);
      break;
    default:
      // Eventos não tratados – loga e retorna OK
      console.log(`Unhandled event: ${eventType}`);
  }
  return NextResponse.json({ received: true, eventType, deliveryId });
} catch (err) {
  console.error(`Webhook processing error [${deliveryId}]:`, err);
  return NextResponse.json(
    { error: 'Processing failed', deliveryId },
    { status: 500 }
  );
}
}

async function handlePush(payload: any) {
  // Atualiza índice de código (code-indexer) com novos arquivos
  const repo = payload.repository.full_name;
  const ref = payload.ref; // refs/heads/main
  const commits = payload.commits;

  console.log(`Push to ${repo}:${ref} with ${commits.length} commits`);
  // TODO: chamar code-indexer.reindex(repo, ref, commits)
}

async function handlePullRequest(payload: any) {
  const action = payload.action;
  const pr = payload.pull_request;
  const repo = payload.repository.full_name;

  if (action === 'closed' && pr.merged) {
    // PR merged – atualiza RefactorSuggestion correspondente
    // O PR pode ter sido aberto pelo ZéCode ou por humano
    // Identificamos via commit message [ze-code-automated]
    const suggestion = await prisma.refactorSuggestion.findFirst({
      where: { prNumber: pr.number, status: 'pr_open' }
    });
    if (suggestion) {
      await prisma.refactorSuggestion.update({
        where: { id: suggestion.id },
        data: { status: 'applied', appliedAt: new Date() }
      });
      console.log(`Suggestion ${suggestion.id} marked as applied (PR #${pr.number} merged)`);
    }
  }
}

async function handleCheckSuite(payload: any) {
  const action = payload.action;
```

```
if (action === 'completed') {
  const suite = payload.check_suite;
  console.log(`Check suite ${suite.conclusion} for ${suite.head_sha}`);
  // Se falhou e foi triggered por PR do ZéCode, notifica admin
}
}

async function handleIssue(payload: any) {
  // Sincroniza labels de issues criadas pelo ZéCode
  console.log(`Issue ${payload.action}: #${payload.issue?.number}`);
}
```

8.2. Configuração do Webhook no GitHub

Passo a passo para registrar o webhook:

1. No repositório: Settings → Webhooks → Add webhook
2. Payload URL:
https://zcc.seuzella.com/api/zcc/github/webhook
3. Content type:
application/json
4. Secret:
[Gere: openssl rand -hex 32]
→ Use o mesmo valor em GITHUB_WEBHOOK_SECRET no .env do ZCC
5. SSL verification:
☒ Enable SSL verification (obrigatório em produção)
6. Which events would you like to trigger this webhook?
☒ Let me select individual events → selecione:
 - ☒ Pull requests
 - ☒ Pushes
 - ☒ Check suites
 - ☒ Check runs
 - ☒ Issues
 - ☒ Issue comments(NÃO selecione "Everything" – risco de volume excessivo)
7. Active: ☒
8. Add webhook
9. Teste: GitHub envia um "ping" automaticamente.
Veja em "Recent Deliveries" se retornou 200 OK.

9. Fase 6: Mirror de Conta Corporativa

O mirror de conta é a migração de todos os commits de um repositório antigo (conta pessoal) para um novo repositório em conta corporativa, reescrevendo o autor/email para refletir a identidade corporativa. Isso preserva o histórico técnico (commits, branches, tags) mas atualiza a autoria para refletir a nova realidade organizacional.

9.1. Ferramenta: git-filter-repo

O git-filter-repo é a ferramenta moderna recomendada pelo Git para reescrita de histórico (sucessor oficial do git filter-branch). É mais rápido, mais seguro, e mantém um mapa de alterações para auditoria. Instalação: `pip install git-filter-repo` ou `apt install git-filter-repo`.

9.2. Script de Migração Robusto

```
#!/bin/bash
# scripts/migrate-to-corporate.sh
# Uso: ./migrate-to-corporate.sh <repo-clone-url> <new-remote-url> <mailmap-file>
#
# Exemplo:
# ./migrate-to-corporate.sh \
#   https://github.com/usuario-antigo/SmartHotel_Zehla.git \
#   https://github.com/empresa-corporativa/SmartHotel_Zehla.git \
#   ./mailmap.txt

set -euo pipefail

OLD_REMOTE="${1:?Uso: $0 <repo-clone-url> <new-remote-url> <mailmap-file>}"
NEW_REMOTE="${2:?}"
MAILMAP="${3:?}"

# Verifica dependências
command -v git-filter-repo >/dev/null 2>&1 || {
    echo "ERRO: git-filter-repo não instalado. Instale com:"
    echo "  pip install git-filter-repo"
    echo "  # ou"
    echo "  apt install git-filter-repo"
    exit 1
}

# Verifica que mailmap existe
[ -f "$MAILMAP" ] || {
    echo "ERRO: Arquivo mailmap não encontrado: $MAILMAP"
    exit 1
}

# Diretórios
TIMESTAMP=$(date +%Y%m%d-%H%M%S)
WORK_DIR="/tmp/migrate-$TIMESTAMP"
BACKUP_DIR="/tmp/backup-$TIMESTAMP"

echo "=== Migração de Conta Corporativa ==="
echo "Repo antigo: $OLD_REMOTE"
echo "Repo novo:   $NEW_REMOTE"
echo "Mailmap:     $MAILMAP"
echo "Work dir:    $WORK_DIR"
echo ""

# 1. Backup absoluto antes de tudo
echo "[1/6] Criando backup de segurança em $BACKUP_DIR ..."
git clone --mirror "$OLD_REMOTE" "$BACKUP_DIR"
echo "  Backup OK ($(du -sh $BACKUP_DIR | cut -f1))"

# 2. Clone de trabalho
echo "[2/6] Clonando repositório para reescrita ..."
git clone --mirror "$OLD_REMOTE" "$WORK_DIR"
cd "$WORK_DIR"

# 3. Copia mailmap para dentro do repo
cp "$MAILMAP" .mailmap

# 4. Aplica reescrita
echo "[4/6] Aplicando reescrita de histórico ..."
git filter-repo --mailmap .mailmap --force

# 5. Verificação
echo "[5/6] Verificando reescrita ..."
echo ""
```

```
echo "=== Commits por autor após reescrita ==="
git log --all --format='%an <ae>' | sort | uniq -c | sort -rn | head -20
echo ""

# Verifica se ainda há commits com email antigo
OLD_EMAILS=$(git log --all --format='%ae' | sort -u | grep -v -E
"empresa\.com|corporativo\.com\.br" || true)
if [ -n "$OLD_EMAILS" ]; then
    echo "AVISO: Alguns commits ainda têm email não-corporativo:"
    echo "$OLD_EMAILS"
    echo ""
    echo "Revise o mailmap e re-execute."
    exit 1
fi

echo "Reescrita verificada – todos os emails são corporativos."

# 6. Push para novo repositório
echo "[6/6] Push para novo repositório corporativo ..."
git remote set-url origin "$NEW_REMOTE"
git push --mirror

echo ""
echo "=== Migração concluída com sucesso! ==="
echo "Backup disponível em: $BACKUP_DIR"
echo "Novo repositório: $NEW_REMOTE"
```

9.3. Exemplo de Arquivo .mailmap

```
# .mailmap
# Formato: Novo Nome <novo@email.com> Nome Antigo <velho@email.com>
#
# Cada linha mapeia um email antigo para o novo.
# Pode mapear múltiplos emails antigos para o mesmo novo.

Marcio Cau <marcio.cau@empresa.com.br> Marcio Cau <marcio@personal-gmail.com>
Marcio Cau <marcio.cau@empresa.com.br> Marcio <marcio.cau@old-account.com>

# Se mudou de nome também:
Marcio Cau <marcio.cau@empresa.com.br> M Cau <mcau@hotmail.com>

# Outros colaboradores que migraram:
João Silva <joao.silva@empresa.com.br> João Silva <joao@gmail.com>
Maria Santos <maria.santos@empresa.com.br> Maria <maria.santos@outlook.com>
```

PARTE III

Operação

10. Troca e Rotação de Credenciais

A troca de credenciais é uma operação crítica que deve ocorrer sem downtime. Cenários típicos: (1) rotação periódica por segurança (a cada 90 dias), (2) suspeita de vazamento (revogação imediata), (3) troca de conta GitHub pessoal → corporativa. Em todos os casos, o fluxo é o mesmo: criar nova credencial → validar → marcar antiga como inativa → opcionalmente revogar via GitHub API.

10.1. Procedimento Passo a Passo

CENÁRIO: Trocar PAT existente "ze-code-prod-corporate" por um novo (rotação periódica ou resposta a incidente)

PRÉ-REQUISITOS:

- Acesso admin ao ZCC
- Acesso ao GitHub com a conta corporativa
- GODMODE habilitado no ZCC (operação sensível)

PASSO 1: Gerar novo PAT no GitHub

- Acesse <https://github.com/settings/personal-access-tokens/new>
- Mesmas permissões do PAT atual (consulte Vault para verificar)
- Mesma expiração (90 dias recomendado)
- Nome: "ze-code-prod-corporate-v2-{YYYYMMDD}"
- Gere e COPIE o token imediatamente

PASSO 2: Validar novo PAT (opcional, via API)

```
curl -H "Authorization: Bearer github_pat_NOVO" \
  https://api.github.com/user
```

→ Deve retornar 200 + JSON com login

PASSO 3: Inserir novo PAT no Vault

- ZCC → Settings → GitHub Credentials
- Selecione credencial antiga "ze-code-prod-corporate"
- Click "Rotate"
- Cole novo PAT
- Click "Validate & Rotate"

O Vault executa:

- Valida novo PAT (chama /user)
- Verifica que permissões são iguais às declaradas
- Cria novo registro GitHubCredential (label sufixada com "-rotated-{ts}")
- Marca antigo como isActive=false (NÃO deleta – histórico)
- Tenta revogar PAT antigo via GitHub API
- Emite evento pat:rotated no AlertBus
- Loga PatAuditLog (action: ROTATE)

PASSO 4: Verificar

- Liste credenciais ativas – deve haver apenas 1 ativa
- PatAuditLog deve mostrar ROTATE com success=true
- Teste executando uma operação: gap-detector run

PASSO 5: Cleanup (opcional, após 7 dias)

- Após confirmar que tudo funciona, delete registros inativos antigos:
ZCC → Settings → GitHub Credentials → [Show Inactive]
→ Select old → "Hard Delete"
- Recomendado: manter pelo menos 30 dias antes de hard delete (para fins de auditoria em caso de incidente pós-rotação)

10.2. Rotação de Emergência (Suspeita de Vazamento)

SE O PAT VAZOU: Execute imediatamente os passos abaixo. Cada minuto conta. Com um PAT vazado, um atacante pode commitar código malicioso, abrir PRs, exfiltrar código, alterar workflows de CI/CD para comprometer supply chain.

RESPOSTA A INCIDENTE – PAT VAZADO

PASSO 1 (imediato, <1 min): Revogar PAT no GitHub

- Acesse <https://github.com/settings/tokens>
- Encontre o PAT suspeito
- Click "Revoke" (NÃO "Delete" – Revoke mantém registro de revogação)

- Confirme

PASSO 2 (<2 min): Marcar como revogado no Vault

- ZCC → Settings → GitHub Credentials
- Selecione o PAT comprometido
- Click "Emergency Revoke"
- Marca isActive=false + audit log

PASSO 3 (<5 min): Auditar ações recentes

- ZCC → Settings → Audit Logs → Filter: credentialId=COMPROMETIDO
- Verifique as últimas 24h de API_CALL
- Procure por: commits suspeitos, PRs não autorizados, alteração em .github/workflows/, alteração em package.json (dependências)

PASSO 4 (<30 min): Forense no GitHub

- GitHub → Settings → Security log
- Filtre por IP suspeito, horário suspeito
- Identifique commits/PRs não autorizados → revert via git revert
- Se workflows foram alterados, restore da branch main anterior

PASSO 5 (pós-incidente): Novo PAT + rotação completa

- Siga procedimento normal de rotação (seção 10.1)
- Documente o incidente em /docs/security/incidents/
- Notifique equipe via canal Slack #security-incidents

PASSO 6 (prevenção):

- Ative alerta "Secret Scanning" no repositório
- Configure GitHub Push Protection (bloqueia commit de tokens)
- Revise todos os .env, configs, scripts em busca do token vazado

10.3. Auto-Rotação Programada

Para reduzir risco operacional, o Vault pode executar rotações programadas via cron. Configuração recomendada: alerta a 14 dias antes da expiração (notifica admin via AlertBus), auto-revocação na data de expiração (se admin não agir).

```
// Cron job diário (já existente em src/lib/cron/)
// Adicionar verificação de PATs expirando

async function checkPatExpiry() {
  const soon = new Date(Date.now() + 14 * 24 * 60 * 60 * 1000); // 14 dias
  const expiringSoon = await prisma.gitHubCredential.findMany({
    where: {
      isActive: true,
      expiresAt: { lte: soon, gt: new Date() }
    }
  });

  for (const cred of expiringSoon) {
    const daysLeft = Math.ceil((cred.expiresAt.getTime() - Date.now()) /
      (24*60*60*1000));
    await alertBus.send({
      severity: 'WARN',
      title: `PAT "${cred.label}" expira em ${daysLeft} dias`,
      message: `Acesse ZCC → Settings → GitHub Credentials para rotacionar.`,
      actionUrl: '/zcc/settings#github-credentials'
    });
    await auditLog(cred.id, 'EXPIRE_ALERT', { success: true });
  }

  // Auto-revoga expirados
  const expired = await prisma.gitHubCredential.findMany({
    where: { isActive: true, expiresAt: { lt: new Date() } }
  });
}
```

```
});  
for (const cred of expired) {  
  await prisma.gitHubCredential.update({  
    where: { id: cred.id },  
    data: { isActive: false }  
  });  
  await alertBus.send({  
    severity: 'CRITICAL',  
    title: `PAT "${cred.label}" EXPIROU e foi revogado`,  
    message: `Operações GitHub falharão até novo PAT ser cadastrado.`,  
    actionUrl: '/zcc/settings#github-credentials'  
  });  
  await auditLog(cred.id, 'REVOKE', {  
    success: true,  
    errorMessage: 'Auto-revoked due to expiration'  
  });  
}  
}
```

11. Checklist de Implantação

Use este checklist para validar que todas as fases foram implementadas e configuradas corretamente antes de considerar o ZéCode em produção.

11.1. Pré-Implantação (Infraestrutura)

Item	Status
GITHUB_PAT_MASTER_KEY gerada (32 bytes base64) e em .env do servidor	☒
ZCC_PUBLIC_URL definida (URL pública HTTPS)	☒
ZE_CODE_WEBHOOK_TOKEN gerado (64 hex) e guardado	☒
Backup completo do banco Prisma antes de migration	☒
HTTPS/TLS válido no endpoint /api/zcc/github/webhook	☒
Firewall permite tráfego inbound das faixas IP do GitHub	☒

11.2. Fase 1 — PAT Vault

Item	Status
Prisma migration adiciona models GitHubCredential e PatAuditLog	☒
migration aplicada: prisma migrate deploy	☒
src/lib/github/pat-vault.ts implementado e testado	☒
Teste unitário: encryptPat → decryptPat retorna original	☒
Teste unitário: master key errada → exceção lançada	☒
Teste unitário: fingerprintPat é determinístico	☒
Admin UI: tela de listagem de credenciais	☒
Admin UI: formulário "Add credential"	☒
Admin UI: botão "Rotate" com confirmação	☒
Admin UI: botão "Emergency Revoke"	☒
Admin UI: tabela de PatAuditLog com filtros	☒

11.3. Fase 2 — GitHub Client

Item	Status
src/lib/github/github-client.ts implementado	☑
Teste: validatePat retorna login correto	☑
Teste: createBranch cria branch com SHA correto	☑
Teste: commitFiles cria commit atômico com múltiplos arquivos	☑
Teste: createPR abre PR com body estruturado	☑
Teste: rate limiting respeitado (X-RateLimit-Remaining=0 pausa)	☑
Teste: retry com backoff em 429/503	☑
Teste: audit log registrado para cada chamada de API	☑

11.4. Fase 3 — Evolve-to-PR

Item	Status
src/lib/cerebro/ze-code/apply-via-pr.ts implementado	☑
API /api/zcc/ze-code/apply com GODMODE protection	☑
Teste: dry-run mode retorna plano sem criar PR	☑
Teste: live mode cria branch + commit + PR	☑
Teste: limite MAX_OPEN_PRS=3 respeitado	☑
Teste: RefactorSuggestion status atualizado para pr_open	☑
Teste: issue linked criada quando solicitado	☑
Commit message inclui [ze-code-automated] e Co-authored-by	☑

11.5. Fase 4 — GitHub Action

Item	Status
.github/workflows/ze-code-review.yml criado	☑
Secret ZE_CODE_API_URL configurado no GitHub	☑
Secret ZE_CODE_WEBHOOK_TOKEN configurado no GitHub	☑
Teste: abertura de PR dispara workflow	☑
Teste: workflow pula PRs do próprio ZéCode (evita loop)	☑
Teste: diff > 500KB é pulado com warning	☑
Teste: review é postada como comentário no PR	☑
Teste: timeout de 5 min não falha o build do PR	☑

11.6. Fase 5 — Webhook Listener

Item	Status
/api/zcc/github/webhook implementado	☑
GITHUB_WEBHOOK_SECRET em .env	☑
Validação de HMAC SHA-256 funcionando	☑
IP allowlist (faixas GitHub) configurada	☑
Webhook registrado no GitHub (Settings → Webhooks)	☑
Teste: ping inicial retorna 200	☑
Teste: push event atualiza índice de código	☑
Teste: PR merged fecha RefactorSuggestion	☑
Idempotência: X-GitHub-Delivery deduplicado	☑

11.7. Fase 6 — Mirror de Conta

Item	Status
git-filter-repo instalado	☑
scripts/migrate-to-corporate.sh criado	☑
Arquivo .mailmap com mapeamento de emails	☑
Backup mirror clone criado antes da migração	☑
Migração executada em repo de teste (não produção)	☑
Verificação pós-migração: 0 commits com email antigo	☑
Push --mirror para novo repositório corporativo	☑
Novo repositório tem branches, tags, PRs antigos preservados	☑
PatVault reconfigurado para apontar para novo repo	☑

11.8. Pós-Implantação

Item	Status
Branch protection ativada em main (min 2 reviews, requires CI)	☑
CODEOWNERS configurado para paths críticos	☑
Secret Scanning + Push Protection habilitados	☑
Dependabot security updates habilitados	☑
Cron job diário checkPatExpiry configurado	☑
Alerta Slack/Teams para eventos critical do PatAuditLog	☑
Documentação interna atualizada (links para esta Bíblia)	☑
Treinamento da equipe: como inserir/rotacionar PAT	☑

12. Troubleshooting

Guia rápido para diagnóstico e correção dos problemas mais comuns. Para cada sintoma, listamos causa provável e solução.

Sintoma	Causa Provável	Solução
Erro 401 Unauthorized em chamada API	PAT revogado ou expirado	Gere novo PAT, rotacione no Vault (seção 10.1)
Erro 403 Forbidden em /repos/...	PAT não tem permissão para o repo	Recrie PAT marcando o repositório (Fine-Grained) ou instale App no repo
Erro 404 em /repos/...	PAT não tem acesso ao repo OU repo não existe	Verifique owner/repo (case-sensitive). Teste com curl primeiro.
Erro 429 Too Many Requests	Rate limit excedido (5000/h)	Aguarde reset (cheça X-RateLimit-Reset). Considere migrar para GitHub App (15000/h).
Webhook retorna 401	Assinatura HMAC inválida (secret errado)	Verifique GITHUB_WEBHOOK_SECRET no .env = secret configurado no GitHub
Webhook retorna 403	IP não está na allowlist	Verifique IP do GitHub em https://api.github.com/meta . Atualize GITHUB_IP_RANGES.
Webhook não chega no servidor	HTTPS inválido ou firewall bloqueando	Teste com https://ping.github.com . Verifique logs do nginx/caddy.
GitHub Action falha em "Call ZéCode API"	ZE_CODE_API_URL inacessível ou token errado	Verifique secret no GitHub. Teste curl da sua máquina.
GitHub Action em loop infinito	ZéCode comentando em PR próprio	Verifique step "Check if ZéCode-automated PR" no workflow
commitFiles cria commit mas PR falha	Branch não existe ou base está desatualizada	Force update branch: <code>git push origin branch --force</code> . Recrie PR.
Migration Prisma falha	Tabela já existe ou coluna conflita	<code>prisma migrate resolve --apply</code> ou <code>rollback manual</code>
PatVault: "Master key must be 32 bytes"	GITHUB_PAT_MASTER_KEY mal formatada	Gere: <code>openssl rand -base64 32</code> . Reinicie servidor.
PatVault: "auth tag invalid"	Ciphertext corrompido ou key trocada	Se key foi trocada, dados antigos são irre recuperáveis. Restaure backup.

13. Próximos Passos (Roadmap M1-M14)

As 6 fases desta Bíblia cobrem a integração GitHub × ZéCode em si. O roadmap abaixo (M1-M14) lista melhorias adicionais que podem ser implementadas após a base estar operacional, ordenadas por prioridade.

13.1. Alta Prioridade (M1-M4)

ID	Título	Descrição	Esforço
M1	Persistir GapFinding/BottleneckFinding no Prisma	Modelos ainda em memória. Adicionar models + migration.	4h
M2	Evolve-to-PR via GitHub API (já coberto na Fase 3)	Aplicar patches reais com auto-criação de branch + PR.	8h
M3	Cron diário de evolução + auto-issue para gaps críticos	Daily job que roda gap-detector e abre issues para severity:critical.	6h
M4	GitHub Action de PR Review (já coberto na Fase 4)	Workflow .github/workflows/ze-code-review.yml.	6h

13.2. Média Prioridade (M5-M8)

ID	Título	Descrição	Esforço
M5	AST Analysis (@babel/parser ou typescript-eslint)	Substituir regex por parsing AST real para análise de código.	12h
M6	TF-IDF hotspot prioritization	Usar code-indexer para priorizar gaps em arquivos mais relevantes.	8h
M7	Confidence calibration via learning-engine	Feedback loop: PRs merged vs rejeitados recalibram confidence.	16h
M8	Suporte a mais linguagens (.py, .sql, .yaml, .sh)	Estender code-reader para além de .ts/.tsx/.js/.jsx.	10h

13.3. Baixa Prioridade (M9-M14)

ID	Título	Descrição	Esforço
M9	Diff viewer interativo no ZCC Panel	Mostrar diff antes/depois com syntax highlighting.	8h
M10	Filtros e busca no painel de Reviews	Filtrar por status, severity, repo, data, author.	4h
M11	Exportação de relatório (PDF/CSV)	Exportar histórico de reviews, gaps, suggestions para análise.	6h
M12	Webhook para Slack/Teams	Notificar canal Slack quando PR é aberto, gap crítico detectado.	4h
M13	Pair programming chat no ZCC	Chat LLM embarcado para discutir sugestões com dev.	20h
M14	ROI metrics dashboard	Calcular horas economizadas, bugs evitados, débito técnico reduzido.	12h

13.4. Ordem de Execução Recomendada

Após completar as 6 fases desta Bíblia (base GitOps), recomendamos a seguinte ordem de execução do roadmap, balanceando valor entregue vs esforço de implementação:

1. M1 — Persistir findings no Prisma (rápido, base para todos os outros)
2. M3 — Cron diário de evolução (entrega valor visível rápido)
3. M8 — Mais linguagens (aumenta cobertura do gap-detector)
4. M5 — AST analysis (qualidade das detecções melhora muito)
5. M6 — TF-IDF prioritization (reduz ruído em projetos grandes)
6. M7 — Confidence calibration (após 100+ suggestions aplicadas)
7. M9-M14 — Em paralelo, conforme demanda da equipe

Conclusão

A Bíblia do ZéCode é um documento vivo. Conforme a integração GitHub × ZCC evolui, novos capítulos serão adicionados: integração com GitHub Codespaces, suporte a GitHub Enterprise Server, GitHub Copilot Extensions, e mais. A arquitetura aqui documentada é flexível o suficiente para absorver essas evoluções sem reescrita arquitetural — apenas adições de novos adaptadores.

O princípio fundamental permanece: segurança primeiro, sempre. Qualquer nova feature que reduza segurança (ex.: conveniência de PAT clássico em vez de Fine-Grained) deve ser rejeitada em code review. O custo de um incidente de segurança GitHub é infinitamente maior que o custo de uma rotação periódica ou de uma configuração inicial mais trabalhosa.

Para questões, sugestões ou correções a este documento, abra uma issue no repositório com label "documentation" e mencione @ze-code-bot.